UTN
Facultad Regional Córdoba

TUP

Tecnicatura Universitaria en Programación

BASES DE DATOS II

Unidad Temática N°1:
Bases de Datos Documentales

Anexo
2° Año – 3° Cuatrimestre

CC BY ND NC

Base de Datos NoSQL: MongoDB **3**

Problema 1.1 3

Problema 1.2 6

Problema 1.3 10

Problema 1.4 16

Problema 1.5 19

BIBLIOGRAFÍA **22**

Base de Datos NoSQL: MongoDB

Problema 1.1

Una biblioteca necesita gestionar los datos de los libros que dispone para consulta y préstamos y se ha decido realizarlo con una base de datos NoSQL. Para ello realice los puntos que siguen:

Cree en MongoDB una base de datos denominada “Biblioteca” y en ella cree la colección “libros” con los siguientes datos:

Título: Harry Potter y la cámara secreta

Editorial: Salamandra

Autor: J. K. Rowling

Género: Fantasía

SubGénero: Novela de aventuras, literatura fantástica y literatura juvenil

Fecha de publicación: 23/11/1999

Número de páginas: 287

ISBN: 9788498380187

Título: Fahrenheit 451

Editorial: Ballantine Books

Autor: Ray Bradbury

Género: Ciencia Ficción

SubGénero: Novela filosófica y novela política

Fecha de publicación: 19/10/1953

Número de páginas: 159

ISBN: 9789875661196

Premios: Premio Hugo y Premio Prometheus

Título: El Ingenioso Hidalgo Don Quijote De La Mancha

Editorial: Juan de la Cuesta

Autor: Miguel de Cervantes

Género: Novela de aventuras

SubGénero: Caballerías realistas

Fecha de publicación: 1605

Número de páginas: 1345

ISBN: 9789873952111

Título: El principito

Editorial: Emecé Editores

Autor: Antoine de Saint-Exupéry

Género: Literatura infantil

SubGénero: Novela filosófica

Fecha de publicación: 20/9/1951

Número de páginas: 111

ISBN: 9789877420746

Título: Crónicas marcianas

Editorial: Minotauro

Autor: Ray Bradbury

Género: Ciencia Ficción

SubGénero: Ficción especulativa y antología, novela

Fecha de publicación: Mayo de 1950

Número de páginas: 272

ISBN: 9789877670295

Título: Yo, robot

Editorial: Gnome Press

Autor: Isaac Asimov

Género: Ciencia Ficción

SubGénero: Cuento

Fecha de publicación: 2 de diciembre de 1950

Número de páginas: 253

ISBN: 9789876283472

Título: El nombre del viento

Editorial: DAW Books

Autor: Patrick Rothfuss

Género: Fantasía

SubGénero: Fantasía heroica, Epopeya

Fecha de publicación: 27/5/2007

Número de páginas: 613

ISBN: 9788401352799

1. Obtenga todos los documentos de la colección del punto anterior.
2. Obtenga todos los documentos en donde el autor sea Ray Bradbury.
3. Obtenga todos los documentos en donde el género sea Ciencia ficción.
4. Obtenga el primer documento en donde el género sea Ciencia ficción
5. Al libro que tiene el título “El principito” añadir el premio “Los 100 mejores libros del siglo”.
6. Al libro que tiene el título “Fahrenheit 451” añadir la sinopsis “Esta colección de relatos reúne la crónica de la colonización de Marte por parte de la Humanidad, que abandona la Tierra en sucesivas oleadas de cohetes plateados y sueña con reproducir en el planeta rojo una civilización de perritos calientes, cómodos sofás y limonada en el porche al atardecer. Pero los colonos también llevan consigo las enfermedades que dispersarán a los marcianos y mostrarán muy poco respeto por una cultura planetaria, misteriosa y fascinante, que éstos intentarán proteger ante la rapacidad de los terrícolas”.

7. Elimine el libro en donde el título sea “El Ingenioso Hidalgo Don Quijote De La Mancha”.
8. Elimine todos los libros en donde el género sea Fantasía.
9. Inserte a la colección “libros” un libro con la siguiente información: Título: 1984, Autor: George Orwell, Editorial: Harvill Secker, Género: Ciencia Ficción, Número de páginas: 326, Premios: Los 100 mejores libros del siglo.

Problema 1.2

Se tiene la base de datos formulaUno, dentro de la colección vehículos tenemos la siguiente estructura de documentos:

```
{
vehiculo_id: 64013221
año: 2021
monoplaza: W12
motor: Mercedes
pilotos:{
nombre: Lewis Hamilton
      nombre: Valtteri Bottas
}
constructor:{
           escudería: Mercedes-AMG Petronas Formula One Team
           sede: Brackley, Reino Unido
           director: Toto Wolff
           }
},
{
vehiculo_id: 61043421
año: 2021
monoplaza: RB16B
motor: Honda
pilotos:{
nombre: Sergio Pérez
      nombre: Max Verstappen
```

```
}
constructor:{
           escudería: Red Bull Racing
           sede: Milton Keynes, Reino Unido
           director: Christian Horner
           }
},
{
vehiculo_id: 63053441
año: 2021
monoplaza: MCL35M
motor: Mercedes
pilotos:{
nombre: Daniel Ricciardo
      nombre: Lando Norris
}
constructor:{
           escudería: McLaren F1 Team
           sede: Woking, Reino Unido
           director: Andreas Seidl
           }
},
{
vehiculo_id: 62073641
año: 2020
monoplaza: RP20
motor: Mercedes
pilotos:{
nombre: Sergio Pérez
      nombre: Lance Stroll
      nombre: Nico Hülkenberg
}
constructor:{
           escudería: BWT Racing Point F1 Team
           sede: Silverstone, Northamptonshire, Reino Unido
           director: Otmar Szafnauer
           }
```

}

1. Escriba una consulta para mostrar todos los documentos de la colección vehículos.

```
db.vehiculos.find();
```

Imagen 1 Elaboración propia

2. Escriba una consulta para mostrar los campos vehiculo_id, monoplaza, motor y año para todos los documentos de la colección vehiculo.

```
db.vehiculos.find({},{"vehiculo_id" : 1,"monoplaza":1,"motor":1,"año" :1});
```

Imagen 2 Elaboración propia

3. Escriba una consulta para mostrar todos los vehículos que poseen motor Mercedes.

```
db.vehiculos.find({"motor": "Mercedes"});
```

Imagen 3 Elaboración propia

4. Escriba una consulta para eliminar el motor del documento con el id 62073641.

```
db.vehiculos.update(
  { vehiculo_id: 62073641},
  {
    $unset:{motor:1}
  })
```

Imagen 4 Elaboración propia

5. Escriba una consulta para añadir al array de pilotos, el piloto con nombre “George Russell” al vehículo con monoplaza igual a W12

```
db.vehiculos.update(
  {
    monoplaza : "W12"
  }, {
    $addToSet : {
      pilotos : "George Russell"
    }
  }
)
```

Imagen 5 Elaboración propia

```
db.vehiculos.update(
    {
        monoplaza : "W12ESC"
    },
    {
        $push :
        {
            pilotos : "George Russell"
        }
    }
  )
```

Imagen 6 Elaboración propia

6. Escriba una consulta para eliminar el piloto con nombre “Nico Hülkenberg” del documento en donde el monoplaza es igual a RP20.

```
db.vehiculos.update(
    {
        monoplaza : "RP20"
    },
    {
        $pull :
        {
            pilotos : "Nico Hülkenberg"
        }
    }
)
```

Imagen 7 Elaboración propia

7. Escriba una consulta para cambiar en todos los documentos el nombre del campo “monoplaza” por “modelo”.

Problema 1.3

Se tiene una colección de artículos de una librería cuyo script se muestra a continuación, se realizan las siguientes acciones:

Script de la colección articulos

```
/*DB: Libreria
COLLECTION: articulos*/

use libreria

db.articulos.insertMany([
{
        "codigo" : 10,
        "descripcion" : "Adhesivo sintético 30 gr",
        "stockMin" : 100,
        "stock" : 10,
        "precio" : 650,
        "observaciones" : "Blanco o transparente"
},
{
        "codigo" : 9,
        "descripcion" : "Carpeta Fantasía - Tamaño Rivadavia",
        "stockMin" : 50,
        "stock" : 150,
        "precio" : 1050
},

{
        "codigo" : 8,
        "descripcion" : "Carpeta fibra negra - Tamaño rivadavia",
        "stockMin" : 50,
        "stock" : 40,
        "precio" : 1500
},

{
        "codigo" : 7,
        "descripcion" : "Separadores tamaño rivadavia * 6 u",
        "stock" : 150,
        "precio" : 960
},

{
        "codigo" : 6,
        "descripcion" : "Lápices Color largos * 12 u",
        "stockMin" : 100,
        "stock" : 500,
        "precio" : 1450,
        "observaciones" : "Marca Faber, Bic o MMM"
},
```

```
{
        "codigo" : 5,
        "descripcion" : "Fibras cortas * 6",
        "stockMin" : 100,
        "stock" : 500,
        "precio" : 1700
},

{
        "codigo" : 4,
        "descripcion" : "Lápices Color cortos  * 12 u",
        "stock" : 150,
        "precio" : 960
},

{
        "codigo" : 3,
        "descripcion" : "Papel p/forrar Araña * 10 u",
        "stockMin" : 100,
        "stock" : 500,
        "precio" : 700,
        "observaciones" : "Color rojo - azul - verde"
},

{
        "codigo" : 2,
        "descripcion" : "Papel p/forrar Fantasía * 10 u",
        "stockMin" : 100,
        "stock" : 500,
        "precio" : 600
},

{
        "codigo" : 1,
        "descripcion" : "Lápiz Evolution HB2 * 4 u",
        "stock" : 150,
        "precio" : 160
},

{ codigo: 11,
  descripcion: 'Block tamaño Rivadavia x 300 hojas',
  marca:"Gloria",
  tipo:["Rallada","Cuadriculadas","Lisas"],
  stockMin: 50,
  stock: 200,
  precio: 2650
},

{codigo: 12,
  descripcion: 'Block tamaño Rivadavia x 300 hojas',
  marca:"Rivadavia",
  tipo:["Rallada","Cuadriculadas","Lisas"],
  stockMin: 50,
  stock: 200,
  precio: 3100
},

{codigo: 13,
  descripcion: {
               producto:'Block tamaño Rivadavia',
```

```
                'tamaño': 100,
                medida: 'hojas',
                marca:"Rivadavia"
                },
  stockMin: 50,
  stock: 200,
  precio: 2000
},

{codigo: 14,
  descripcion: {producto:'Lapiz negro', tamaño: 10, medida:
'unidades',marca:"Faber"},
  stockMin: 50,
  stock: 200,
  precio: 1000
},

{codigo: 15,
  descripcion: 'Cajas de cartón',
  tamaño: [10,15,25],
  stockMin: 50,
  stock: 200,
  precio: 1500
},

{codigo: 16,
  descripcion: 'Cajas de fibra',
  tamaño: [20,25,30],
  stockMin: 50,
  stock: 200,
  precio: 2000
},

{codigo: 17,
  descripcion: 'Caja archivo',
  tamaño: [30,25,50],
  stockMin: 50,
  stock: 200,
  precio: 2500
}])
```

Acciones:

```
//Consultas:
//Operadores de comparación:
//$eq; $lt, $lte, $gt, $gte, $ne, $in, $nin

//igual
db.articulos.find({codigo:{$eq:10}})

//menor
db.articulos.find({precio:{$lt:1500}})

//menor o igual
db.articulos.find({precio:{$lte:1500}})

//mayor
db.articulos.find({precio:{$gt:1000}})

//mayor o igual
```

```
db.articulos.find({precio:{$gte:1000}})

//no igual
db.articulos.find({marca:{$ne:'Rivadavia'}})

//dentro de
db.articulos.find({codigo:{$in:[1,5,6]}})

// no dentro de
db.articulos.find({codigo:{$nin:[1,5,6]}})

//exists
db.articulos.find({observaciones:{$exists:true}})

//ENTRE: lista todos los articulos con precios mayores o iguales a 1000 y
menores o iguales a 1500
db.articulos.find({precio:{$gte:1000,$lte:1500}})

//OR: listar los artículos cuyo precio sea mayor a 1000 o bien el stock
//sea menor a 100
db.articulos.find({
        $or:[{precio:{$gt:1000}},{stock:{$lt:100}}]
})

//AND: listar los artículos cuyo precio sea mayor a 1000 y que el stock
//sea menor a 100
db.articulos.find({
        $and:[{precio:{$gt:1000}},{stock:{$lt:100}}]})

//Muestre los articulos cuyo precio esté entre 1000 y 3000 o el stock esté
//entre 10 y 100
db.articulos.find({
    $or: [
        { precio: { $gte: 1000, $lte: 3000 } },
        { stock: { $gte: 10, $lte: 100 } }
    ]
})

//Muestre los articulos cuyo precio sea mayor 1000 o el stock menor a 100
o descripción contenga A o empiece con L
db.articulos.find({
    $or: [
        { precio: { $gt: 1000 } },
        { stock: { $lt: 100 } },
        { descripcion: /A/ },
        { descripcion: /^L/ }
    ]
})

//Mostrar solo descripción del artículo con código=1
db.articulos.find({codigo:1},{descripcion:1})

//El _id se muestra para no verlo y mostrar descripción y precio
db.articulos.find({codigo:1},{_id:0,descripcion:1,precio:1})

//Buscar en documentos embebidos
//Buscar la marca Faber dentro del subdocumento descripción

db.articulos.find({'descripcion.marca':'Faber'})

//Buscar la marca Faber y el tamaño mayor a 5 dentro del subdocumento
```

```
//descripción
db.articulos.find({
        'descripcion.marca':'Faber',
        'descripcion.tamaño':{$gt:5}
})

//consultar los articulos con fecha anterior a una fecha determinada
db.articulos.find({fecha:{$lt:new Date(2023,9,5)}})

//Patrón de búsqueda
//Empiece con "B"
db.articulos.find({descripcion:/^B/})

//Termine con "a"
db.articulos.find({descripcion:/a$/})

//contenga "R"
db.articulos.find({descripcion:/R/})

//buscar dentro de un array

//tamaño mayor a 30
db.articulos.find({tamaño:{$gt:30}})

//el segundo tamaño mayor a 30
db.articulos.find({'tamaño.1':{$gt:30}})

//que sea rallado
db.articulos.find({tipo:"Rallada"})

//que sea "Rallada","Cuadriculadas"y"Lisas"
db.articulos.find(
{tipo:["Rallada","Cuadriculadas","Lisas"]})

//Cambiar el nombre de un campo por otro
//ejemplo: stock por stockActual
db.articulos.updateMany({ }, {$rename:{"stock":"stockActual"}})

//
//ACTUALIZACIÓN DE DATOS

//$set
//Modificar el precio de un articulo con codigo igual a 12 y muestre
db.articulos.updateOne({codigo:12},{$set:{precio:2950}})
db.articulos.find({codigo:12})

//Modificar el precio y el stock de un articulo con codigo igual a 12
db.articulos.updateOne(
        {codigo:12},
        {$set:{precio:2500,stock:100}
})
db.articulos.find({codigo:12})

//modifica el campo marca, si no existe el campo lo agrega
db.articulos.updateOne(
        {codigo:1},
        {$set:{marca:"BIC"}})
db.articulos.find({codigo:1})

//$unset
//elimina un campo
```

```
db.articulos.updateOne(
        {codigo:1},
        {$unset:{marca:''}})
db.articulos.find({codigo:1})

//$push: agregar un elemento en un array
db.articulos.updateOne(
        {codigo:12},
        {$push:{tipo:"otro"}})

db.articulos.find({codigo:{$eq:12}})

//$pull: elimina un elemento de un array
db.articulos.updateOne(
        {codigo:12},
        {$pull:{tipo:'otro'}})

db.articulos.find({codigo:{$eq:12}})

//
//ELIMINAR DOCUMENTOS

//elimina el primero que cumple la condición
db.articulos.deleteMany({codigo : {$eq : 10 }})

//elimina todos los que cumplen la condición
db.articulos.deleteMany({precio : {$lte : 50 }})

//elimina todo
db.articulos.deleteMany({})

//datos de tipo DATE
//Insertar un documento con la fecha del sistema
db.articulos.insertOne({codigo: 15,
  descripcion: 'Escuadra 90°',
  stockMin: 50,
  stock: 200,
  precio: 100,
  fecha: new Date()})

db.articulos.find({codigo:{$eq:15}})

//Insertar un documento con la fecha como año,mes-1,día
db.articulos.insertOne({codigo: 16,
  descripcion: 'Transportador',
  stockMin: 50,
  stock: 200,
  precio: 100,
  fecha:new Date(2023,9,1)})

db.articulos.find({codigo:{$gte:15}})

//ELIMINAR LA BASE DE DATOS
use librería
db.dropDatabase()
```

Problema 1.4

Se va a trabajar con una nueva colección llamada clientes, que representa personas registradas en una empresa de ventas de tecnología.

Crear la base de datos y la colección con algunos documentos iniciales

```
use empresa_tecnologia

db.clientes.insertMany([
  {
    nombre: "Carlos Pérez",
    edad: 34,
    correo: "carlos.perez@gmail.com",
    direccion: { ciudad: "Córdoba", calle: "Belgrano 123" },
    compras: ["notebook", "mouse"],
    activo: true
  },
  {
    nombre: "Lucía Fernández",
    edad: 28,
    correo: "lucia.fernandez@hotmail.com",
    direccion: { ciudad: "Rosario", calle: "San Martín 456" },
    compras: ["impresora"],
    activo: false
  },
  {
    nombre: "Martín Gómez",
    edad: 45,
    correo: "martin.gomez@yahoo.com",
    direccion: { ciudad: "Mendoza", calle: "Sarmiento 789" },
    compras: ["monitor", "teclado", "webcam"],
    activo: true
  }
])
```

Consultas simples

1. Mostrar todos los clientes registrados

```
db.clientes.find()
```

Devuelve todos los documentos en la colección.

2. Mostrar sólo el nombre y ciudad de cada cliente

```
db.clientes.find({}, { nombre: 1, "direccion.ciudad": 1, _id: 0 })
```

Se proyectan sólo los campos deseados (sin _id).

3. Buscar todos los clientes activos

```
db.clientes.find({ activo: true })
```

Filtra los documentos cuyo campo activo es true.

Consultas con condiciones

4. Buscar clientes mayores de 30 años

```
db.clientes.find({ edad: { $gt: 30 } })
```

Utiliza el operador $gt (mayor que) para filtrar por edad.

5. Buscar clientes de Córdoba o Mendoza

```
db.clientes.find({ "direccion.ciudad": { $in: ["Córdoba", "Mendoza"] } })
```

Usa $in para buscar coincidencias múltiples en arrays.

Consultas con arrays

6. Mostrar clientes que hayan comprado "monitor"

```
db.clientes.find({ compras: "monitor" })
```

Consulta que busca si "monitor" está en el array compras.

7. Mostrar clientes que hayan hecho exactamente 2 compras

```
db.clientes.find({ compras: { $size: 2 } })
```

El operador $size permite filtrar por longitud del array.

Actualizaciones

8. Cambiar la ciudad de residencia de "Lucía Fernández" a "Buenos Aires"

```
db.clientes.updateOne(
  { nombre: "Lucía Fernández" },
  { $set: { "direccion.ciudad": "Buenos Aires" } }
)
```

Modifica un campo específico de un documento.

9. Agregar una nueva compra a "Carlos Pérez"

```
db.clientes.updateOne(
  { nombre: "Carlos Pérez" },
  { $push: { compras: "auriculares" } }
)
```

Agrega un elemento al array compras.

Eliminación

10. Eliminar a los clientes que no están activos

```
db.clientes.deleteMany({ activo: false })
```

Elimina todos los documentos con activo: false.

Búsqueda con condiciones combinadas

11. Buscar clientes activos que vivan en Córdoba y hayan comprado "mouse"

```
db.clientes.find({
  activo: true,
  "direccion.ciudad": "Córdoba",
  compras: "mouse"
})
```

Se combinan varias condiciones en una sola consulta.

Actualizaciones avanzadas

12. Aumentar en 1 la edad de todos los clientes activos

```
db.clientes.updateMany(
  { activo: true },
  { $inc: { edad: 1 } }
)
```

$inc incrementa numéricamente un campo.

Ordenamientos y límites

13. Listar los 2 clientes más jóvenes

```
db.clientes.find().sort({ edad: 1 }).limit(2)
```

Ordena por edad ascendente y muestra sólo los 2 primeros.

Proyección y renombre

14. Mostrar solo nombre y correo, pero renombrando "correo" a "email"

```
db.clientes.aggregate([
  { $project: { _id: 0, nombre: 1, email: "$correo" } }
])
```

Usa aggregate y $project para renombrar campos en el resultado.

Conteo

15. Contar cuántos clientes hay en total

```
db.clientes.countDocuments({})
```

Devuelve el total de documentos en la colección.

Problema 1.5

Se continúa con la misma colección utilizada en la guía anterior

1. Operadores lógicos combinados

a) Buscar clientes que vivan en Córdoba o tengan más de 40 años

```
db.clientes.find({
  $or: [
    { "direccion.ciudad": "Córdoba" },
    { edad: { $gt: 40 } }
  ]
})
```

Utiliza $or para combinar condiciones.

2: Buscar clientes activos y que hayan comprado más de un producto

```
db.clientes.find({
  $and: [
    { activo: true },
    { "compras.1": { $exists: true } }
  ]
})
```

El índice .1 indica que al menos hay dos elementos (posición 0 y 1).

Búsqueda entre rangos

3: Buscar clientes cuya edad esté entre 30 y 40 (inclusive)

```
db.clientes.find({
  edad: { $gte: 30, $lte: 40 }
})
```

$gte (mayor o igual) y $lte (menor o igual).

Expresiones regulares

4: Buscar clientes cuyo correo sea de Gmail

```
db.clientes.find({
  correo: { $regex: /@gmail\.com$/ }
```

```
})
```

El $regex permite buscar textos con patrones definidos.

5: Buscar clientes cuyo nombre comience con "M"

```
db.clientes.find({
  nombre: { $regex: /^M/, $options: "i" }
})
```

^M busca al comienzo; la opción "i" ignora mayúsculas/minúsculas.

Documentos con fechas

Supongamos que agregamos una fecha de registro a los clientes:

6: Agregar una fecha de registro a los clientes existentes

```
db.clientes.updateMany({}, { $set: { fecha_registro: new Date("2023-01-
01") } })
```

Se agrega un campo de tipo Date a todos los documentos.

7: Buscar clientes registrados después del 1 de junio de 2023

```
db.clientes.find({
  fecha_registro: { $gt: new Date("2023-06-01") }
})
```

Permite trabajar con filtros de fechas.

Agregaciones

8: Obtener el promedio de edad de todos los clientes activos

```
db.clientes.aggregate([
  { $match: { activo: true } },
  { $group: { _id: null, promedio_edad: { $avg: "$edad" } } }
])
Agrupa documentos y calcula promedio.
```

9: Contar cuántos clientes hay por ciudad

```
db.clientes.aggregate([
  { $group: { _id: "$direccion.ciudad", cantidad: { $sum: 1 } } }
])
```

Agrupa por ciudad y cuenta los registros.

10: Listar las compras más frecuentes realizadas por los clientes (desnormalizando arrays)

```
db.clientes.aggregate([
  { $unwind: "$compras" },
  { $group: { _id: "$compras", cantidad: { $sum: 1 } } },
  { $sort: { cantidad: -1 } }
])
```

$unwind desarma los arrays para permitir el conteo individual.

Pipeline completo con filtrado, proyección y ordenamiento

11: Mostrar los nombres y edades de los clientes activos, ordenados de mayor a menor edad

```
db.clientes.aggregate([
  { $match: { activo: true } },
  { $project: { _id: 0, nombre: 1, edad: 1 } },
  { $sort: { edad: -1 } }
])
```

Ejemplo completo combinando varias etapas del pipeline de agregación.

BIBLIOGRAFÍA

Gorman K., Hirt A., Noderer D., Rowland-Jones J., Sirpal A., Ryan D. & Woody B (2019) Introducing Microsoft SQL Server 2019. Reliability, scalability, and security both on premises and in the cloud. Packt Publishing Ltd. Birmingham UK

Microsoft. SQL Server 2016. Disponible en: https://www.microsoft.com/es-es/sql-server/sql-server-2016

Opel, A. & Sheldon, R. (2010). Fundamentos de SQL. Madrid. Editorial Mc Graw Hill

Varga S., Cherry D., D'Antoni J. (2016). Introducing Microsoft SQL Server 2016 Mission-Critical Applications, Deeper Insights, Hyperscale Cloud. Washington. Microsoft Press

Atribución-No Comercial-Sin Derivadas

Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Referenciarlo de la siguiente manera: Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.